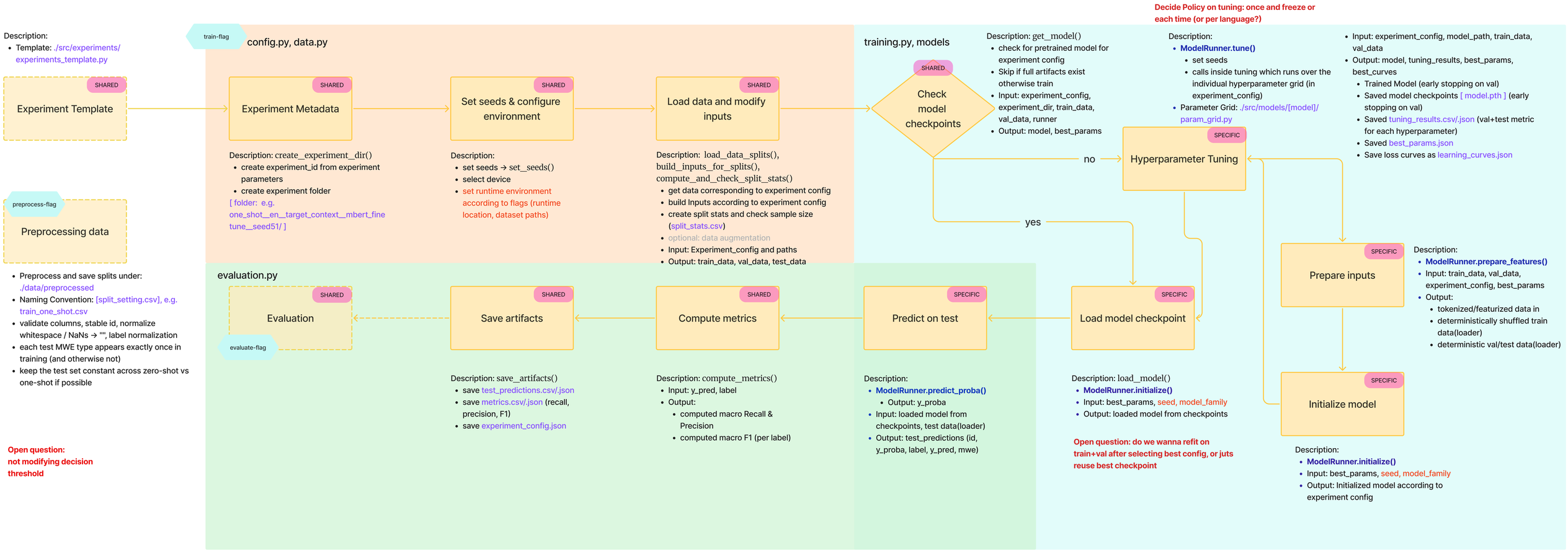
Description:
Template: ./src/experiments/experiments_template.py
SHARED
Experiment Template
preprocess-flag
Preprocessing data
Preprocess and save splits under: ./data/preprocessed
Naming Convention: [split_setting.csv], e.g. train_one_shot.csv
validate columns, stable id, normalize whitespace / NaNs → "", label normalization
each test MWE type appears exactly once in training (and otherwise not)
keep the test set constant across zero-shot vs one-shot if possible
Open question: not modifying decision threshold
train-flag
config.py, data.py
SHARED
Experiment Metadata
Description: create_experiment_dir()
create experiment_id from experiment parameters
create experiment folder
[folder: e.g. one_shot__en__target_context__mbert_fine tune__seed51/]
SHARED
Set seeds & configure environment
Description:
set seeds → set_seeds()
select device
set runtime environment according to flags (runtime location, dataset paths)
SHARED
Load data and modify inputs
Description: load_data_splits(), build_inputs_for_splits(), compute_and_check_split_stats()
get data corresponding to experiment config
build Inputs according to experiment config
create split stats and check sample size (split_stats.csv)
optional: data augmentation
Input: Experiment_config and paths
Output: train_data, val_data, test_data
training.py, models
SHARED
Check model checkpoints
Description: get_model()
check for pretrained model for experiment config
Skip if full artifacts exist otherwise train
Input: experiment_config, experiment_dir, train_data, val_data, runner
Output: model, best_params
no
yes
Decide Policy on tuning: once and freeze or each time (or per language?)
SPECIFIC
Hyperparameter Tuning
Description:
ModelRunner.tune()
set seeds
calls inside tuning which runs over the individual hyperparameter grid (in experiment_config)
Parameter Grid: ./src/models/[model]/param_grid.py
Input: experiment_config, model_path, train_data, val_data
Output: model, tuning_results, best_params, best_curves
Trained Model (early stopping on val)
Saved model checkpoints [model.pth] (early stopping on val)
Saved tuning_results.csv/.json (val+test metric for each hyperparameter)
Saved best_params.json
Save loss curves as learning_curves.json
SPECIFIC
Prepare inputs
Description:
ModelRunner.prepare_features()
Input: train_data, val_data, experiment_config, best_params
Output:
tokenized/featurized data in
deterministically shuffled train data(loader)
deterministic val/test data(loader)
SPECIFIC
Initialize model
Description:
ModelRunner.initialize()
Input: best_params, seed, model_family
Output: Initialized model according to experiment config
SPECIFIC
Load model checkpoint
Description: load_model()
ModelRunner.initialize()
Input: best_params, seed, model_family
Output: loaded model from checkpoints
Open question: do we wanna refit on train+val after selecting best config, or juts reuse best checkpoint
SPECIFIC
Predict on test
Description:
ModelRunner.predict_proba()
Output: y_proba
Input: loaded model from checkpoints, test data(loader)
Output: test_predictions (id, y_proba, label, y_pred, mwe)
evaluation.py
SHARED
Compute metrics
Description: compute_metrics()
Input: y_pred, label
Output:
computed macro Recall & Precision
computed macro F1 (per label)
SHARED
Save artifacts
Description: save_artifacts()
save test_predictions.csv/.json
save metrics.csv/.json (recall, precision, F1)
save experiment_config.json
SHARED
Evaluation
evaluate-flag